T3000 API & Protocol

Modbus Protocol · built Nov 13, 2026

Modbus Protocol

Modbus TCP/IP and RTU protocol support for legacy devices.

Modbus TCP

Connection:

- IP Address: Device IP
- Port: 502 (standard)
- Protocol: Modbus TCP

Function Codes:

- 0x03: Read Holding Registers
- 0x04: Read Input Registers
- 0x06: Write Single Register
- 0x10: Write Multiple Registers

Modbus RTU

Serial Settings:

- Baud Rate: 9600, 19200, 38400
- Data Bits: 8
- Parity: None, Even, Odd
- Stop Bits: 1, 2

Slave ID Range: 1-247

Register Mapping

Register	Type	Description
1-100	Input	Analog inputs
101-200	Output	Analog outputs
201-300	Variable	Variables

Error Codes

- 0x01: Illegal Function
- 0x02: Illegal Data Address
- 0x03: Illegal Data Value
- 0x04: Slave Device Failure

Modbus Protocol Implementation

Protocol Specifications

Modbus TCP Frame Format

```
TEXT

MBAP Header (7 bytes) + PDU (up to 253 bytes)

[Transaction ID (2)] [Protocol ID (2)] [Length (2)] [Unit ID (1)] [Function Code (1)] [D
```

Modbus RTU Frame Format

```
TEXT

[Slave ID (1)] [Function Code (1)] [Data (n)] [CRC (2)]
```

Complete Register Map

T3000 Modbus Address Mapping

TYPESCRIPT

```typescript
const REGISTER_MAP = {
  // Input Registers (30001-30064)
  inputs: {
    base: 30001,
    count: 64,
    access: 'read-only',
    functionCode: 0x04
  },

  // Output Registers (40001-40032)
  outputs: {
    base: 40001,
    count: 32,
    access: 'read-write',
    functionCode: 0x03  // Read
    // 0x06 for single write, 0x10 for multiple write
  },

  // Variable Registers (40033-40096)
  variables: {
    base: 40033,
    count: 64,
    access: 'read-write',
    functionCode: 0x03
  },

  // Control Parameters (40097-40200)
  control: {
    base: 40097,
    count: 104,
    access: 'read-write',
    functionCode: 0x03
  }
};
```

Data Scaling

TYPESCRIPT

```typescript
// Temperature values are scaled by 10
function scaleTemperature(rawValue: number): number {
  return rawValue / 10.0;
}

function unscaleTemperature(tempValue: number): number {
  return Math.round(tempValue * 10);
}

// Example: Read IN1 temperature
const rawValue = await readInputRegister(30001);
const temperature = scaleTemperature(rawValue);  // 725 -> 72.5°F
```

Modbus TCP Client Implementation

Full-Featured Client

TYPESCRIPT

```typescript
import ModbusRTU from 'modbus-serial';

class T3000ModbusTCPClient {
  private client: ModbusRTU;
  private host: string;
  private port: number;
  private unitId: number;
  private connected = false;

  constructor(host: string, port = 502, unitId = 1) {
    this.client = new ModbusRTU();
    this.host = host;
    this.port = port;
    this.unitId = unitId;
  }

  async connect(): Promise<void> {
    try {
      await this.client.connectTCP(this.host, { port: this.port });
      this.client.setID(this.unitId);
      this.client.setTimeout(5000);
      this.connected = true;
    } catch (error) {
      throw new Error(`Failed to connect: ${error.message}`);
    }
  }

  async readInputs(startRegister: number, count: number): Promise<number[]> {
    this.ensureConnected();

    // Convert from Modbus address to zero-based offset
    const address = startRegister - 30001;

    const result = await this.client.readInputRegisters(address, count);
    return result.data.map(v => scaleTemperature(v));
  }
```

```
  async readHoldingRegisters(startRegister: number, count: number): Promise<number[]> {
    this.ensureConnected();

    const address = startRegister - 40001;
    const result = await this.client.readHoldingRegisters(address, count);
    return result.data.map(v => scaleTemperature(v));
  }

  async writeSingleRegister(register: number, value: number): Promise<void> {
    this.ensureConnected();

    const address = register - 40001;
    const scaledValue = unscaleTemperature(value);

    await this.client.writeRegister(address, scaledValue);
  }

  async writeMultipleRegisters(startRegister: number, values: number[]): Promise<void> {
    this.ensureConnected();

    const address = startRegister - 40001;
    const scaledValues = values.map(v => unscaleTemperature(v));

    await this.client.writeRegisters(address, scaledValues);
  }

  private ensureConnected() {
    if (!this.connected) {
      throw new Error('Not connected. Call connect() first.');
    }
  }

  async disconnect(): Promise<void> {
    await this.client.close();
    this.connected = false;
  }
}
```

Usage Examples

TYPESCRIPT

```typescript
// Initialize client
const modbus = new T3000ModbusTCPClient('192.168.1.100');
await modbus.connect();

// Read all inputs
const inputs = await modbus.readInputs(30001, 64);
console.log('IN1:', inputs[0]);

// Read specific output
const outputs = await modbus.readHoldingRegisters(40001, 1);
console.log('OUT1:', outputs[0]);

// Write to output
await modbus.writeSingleRegister(40001, 75.5);

// Write multiple values
await modbus.writeMultipleRegisters(40001, [75.5, 50.0, 60.0]);

await modbus.disconnect();
```

Modbus RTU Implementation

Serial Port Client

PYTHON

```python
from pymodbus.client import ModbusSerialClient
import struct

class T3000ModbusRTU:
    def __init__(self, port='/dev/ttyUSB0', baudrate=19200, slave_id=1):
        self.client = ModbusSerialClient(
            port=port,
            baudrate=baudrate,
            bytesize=8,
            parity='N',
            stopbits=1,
            timeout=1
        )
        self.slave_id = slave_id

    def connect(self):
        return self.client.connect()

    def read_inputs(self, start_register, count):
        """Read input registers (function code 0x04)"""
        address = start_register - 30001  # Convert to zero-based
        result = self.client.read_input_registers(
            address,
            count,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

        return [value / 10.0 for value in result.registers]

    def read_holding_registers(self, start_register, count):
        """Read holding registers (function code 0x03)"""
        address = start_register - 40001
        result = self.client.read_holding_registers(
```

```
            address,
            count,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

        return [value / 10.0 for value in result.registers]

    def write_register(self, register, value):
        """Write single register (function code 0x06)"""
        address = register - 40001
        scaled_value = int(value * 10)

        result = self.client.write_register(
            address,
            scaled_value,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

    def write_registers(self, start_register, values):
        """Write multiple registers (function code 0x10)"""
        address = start_register - 40001
        scaled_values = [int(v * 10) for v in values]

        result = self.client.write_registers(
            address,
            scaled_values,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

    def close(self):
        self.client.close()
```

Usage Example

PYTHON

```python
## Initialize
modbus = T3000ModbusRTU(port='/dev/ttyUSB0', baudrate=19200, slave_id=1)
modbus.connect()

## Read inputs
inputs = modbus.read_inputs(30001, 10)
print(f"IN1: {inputs[0]}°F")

## Write output
modbus.write_register(40001, 72.5)

modbus.close()
```

Error Handling

Exception Handling

TYPESCRIPT

```typescript
async function robustModbusOperation<T>(
  operation: () => Promise<T>,
  maxRetries = 3,
  delayMs = 1000
): Promise<T> {
  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      // Check for non-recoverable errors
      if (error.modbusCode === 0x02) {  // Illegal data address
        throw error;  // Don't retry
      }

      if (attempt < maxRetries - 1) {
        await new Promise(resolve => setTimeout(resolve, delayMs));
        delayMs *= 2;  // Exponential backoff
      }
    }
  }

  throw lastError!;
}

// Usage
const value = await robustModbusOperation(
  () => modbus.readInputs(30001, 1)
);
```

Performance Optimization

Batch Reading

TYPESCRIPT

```typescript
// ❌ Inefficient: 64 separate reads
for (let i = 0; i < 64; i++) {
  const value = await modbus.readInputs(30001 + i, 1);
  console.log(`IN${i+1}: ${value[0]}`);
}

// ✅ Efficient: 1 batch read
const allInputs = await modbus.readInputs(30001, 64);
allInputs.forEach((value, i) => {
  console.log(`IN${i+1}: ${value}`);
});
```

Connection Pooling

TYPESCRIPT

```typescript
class ModbusConnectionPool {
  private connections: T3000ModbusTCPClient[] = [];
  private available: T3000ModbusTCPClient[] = [];
  private maxConnections: number;

  constructor(host: string, maxConnections = 5) {
    this.maxConnections = maxConnections;
  }

  async acquire(): Promise<T3000ModbusTCPClient> {
    if (this.available.length > 0) {
      return this.available.pop()!;
    }

    if (this.connections.length < this.maxConnections) {
      const client = new T3000ModbusTCPClient(this.host);
      await client.connect();
      this.connections.push(client);
      return client;
    }

    // Wait for available connection
    return new Promise((resolve) => {
      const checkInterval = setInterval(() => {
        if (this.available.length > 0) {
          clearInterval(checkInterval);
          resolve(this.available.pop()!);
        }
      }, 10);
    });
  }

  release(client: T3000ModbusTCPClient): void {
    this.available.push(client);
  }
}
```

Next Steps

- REST API
- WebSocket API
- Events